Full-Duplex Neural Video Coding on Laptop SoCs: Literature Review, Research Gap and Proposed Innovation

Mohammed Alawadhi, Shams Al-Ani, Warda Ul Hasan
Department of Computer Science and Engineering, American University of Sharjah, UAE
b00108492@aus.edu, g00113665@aus.edu, g00090299@aus.edu

***Abstract*—Learned video codecs of the DCVC family now save 21–42% bitrate over H.266/VVC and run far above real time on datacenter GPUs, and MLVC has made them decode reliably on laptop neural processing units (NPUs). Every published speed, however, times the encoder *or* the decoder alone, on one processor, with no energy figure, although a video call needs both at once on one device. We review 25 peer-reviewed studies from 2021–2026 in six methodological groups, compare the DCVC family on a common anchor, and identify the gap: no learned codec has been run full duplex on a consumer device, and the effect of placing its parts on the CPU, GPU and NPU on throughput, latency, energy and decoding correctness is unknown. We propose DuplexNVC, an energy-aware scheduler that pipelines both call directions across an Apple Silicon laptop's CPU, GPU and Neural Engine and selects model and resolution to hold 30 fps each way.**

***Index Terms*—learned video compression, DCVC, MLVC, heterogeneous computing, NPU, energy efficiency, video conferencing**

I. Background and Problem Context (8.1)

Importance. In a video call every device encodes its own camera and decodes the other participants at the same time [12]. Learned codecs save 21–42% bitrate over the VVC reference encoder [10], [11]; MLVC is the first built to run on laptop NPUs from Apple, Intel and Qualcomm [12]. Whether one laptop can run both directions at once, and at what energy, has not been measured.

Challenges. (i) *Computation*: VTM-17 encodes 1080p at 0.01 fps; DCVC-FM codes it at 3.7/4.4 fps (enc/dec) on an RTX 4090 [11]. (ii) *Operational cost*: memory traffic and module count, not MACs, limit speed [10]. (iii) *Determinism*: arithmetic coding needs bit-identical probabilities, and chip-specific floating-point rounding corrupts decoding [12], [15]. (iv) *Concurrency*: encoder and decoder share the accelerator, memory bandwidth and CPU entropy coder. (v) *Energy and thermals* on battery-powered laptops.

Traditional approach. H.264/AVC [1], HEVC [2] and VVC [3] use hand-designed hybrid block coding; real-time laptop calling relies on their fixed-function hardware encoders, which is why MLVC's anchor is Intel hardware HEVC (QSV) [12]. Learned codecs replace each stage with networks trained for rate and distortion [4], [5], [39].

COE 694-07 Parallel Computing for Data-Intensive and AI Applications, Fall 2026. Section titles give the matching deliverable item (8.1–8.9).

Evaluation criteria. BD-rate [32] against an anchor (x265, HM, VTM or hardware HEVC) in PSNR, MS-SSIM [33], VMAF [34] or MOS; encode and decode fps at a stated resolution and device; delay (low-delay vs. chunk/random access); kMACs/pixel and parameters; cross-platform decoding success [12]. No reviewed learned codec reports energy. Training uses Vimeo-90k [37]; testing uses HEVC Class B–E [38], UVG [35] and MCL-JCV [36], low-delay, YUV420 or RGB.

II. Structured Review of Existing Methods (8.2)

We group the literature from hand-designed to learned and from datacenter to device. Groups A–F cover the application domain; group G covers the parallel-computing domain.

A. Classical hybrid block codecs

Idea: block partitioning, intra/motion-compensated prediction, transform, quantization, CABAC [1]–[3]. *Data*: JVET/JCT-VC common test sequences [38]. *Performance*: in low delay HM needs 42.4% more bits than VTM; VTM hierarchical-B saves 33.8% over VTM low-delay; VTM encodes 1080p at 0.01 fps [11]. *Strengths*: standardized, integer-exact decoding, hardware encoders everywhere. *Limitations*: each generation buys its gain with much higher encoder complexity. *Open*: learned codecs now exceed VVC, so deployment of learned codecs is the open question.

B. Learned codecs with explicit motion (2019–2024)

Idea: DVC [4] learned every classical stage; DCVC [5] replaced residual subtraction with *conditional coding* on a feature-domain temporal context. Successors added multi-scale temporal contexts (TCM [6]), hybrid spatial-temporal entropy models with variable rate (HEM [7]), diverse contexts (DC [8]), feature modulation over an 11.4 dB quality range (FM [9]) and bidirectional coding [22], [23]. *Data*: Vimeo-90k; HEVC B–E, UVG, MCL-JCV. *Performance*: DCVC saves 26.0% over x265 veryslow (HEVC B) [5]; DCVC-FM 25.5% over VTM-17 [9]; BiECVC is the first to beat VTM random access on all sets [23]. *Limitations*: DCVC-DC/FM code 1080p at 3.3/4.3 and 5.0/5.9 fps on an A100; DCVC-FM's flow branch has 13× fewer MACs than its coding branch but about half as many conv layers, so many

small layers dominate latency [10]. *Note on frame-difference coding*: coding the difference between consecutive frames, a natural way to adapt effort to content, is exactly residual coding (DVC); DCVC showed that conditioning on a learned temporal context, so the network decides what to transmit, saves more bits than subtracting a prediction [5]. Static, low-texture regions already cost few bits under conditional coding, so the idea is subsumed by the codec we build on.

C. Motion-free real-time learned codecs (2024–2026)

Idea: remove explicit motion and cut operational cost [10]. DCVC-RT [10] uses implicit temporal modelling (3.4× faster encoding for +2.1% BD-rate), a single 1/8-scale latent (about 3.6× faster), module-bank rate control and an int16 mode; it overlaps CPU entropy coding (up to 2.6 ms vs. 7.8 ms inference per 1080p frame) with GPU modules for 12%/9% faster encode/decode. DCVC-UF [11] codes chunks of 8 frames into one latent, decodes them with frame-specific decoders, and reads the bitstream in one step. DHVC 2.0 [17] predicts across scales but needs four pipelined GPUs for real time; UI^{2}C [18] unifies intra and inter coding (−12.1% vs. DCVC-RT). *Performance*: DCVC-RT −21.0% vs. VTM-17 at 125.2/112.8 fps (A100); DCVC-UF HT-L −42.2% at 371.1/273.6 fps and HT-S −31.6% at 655.9/453.3 fps (RTX 4090) [10], [11]. *Limitations*: weak above about 40 dB; int16 over 4× slower than fp16; fixed chunk size and "no cross-chunk pipeline parallelism" [11]; NVIDIA GPUs only.

D. Cross-platform and on-device learned codecs (2022–2026)

Idea: decode correctly and in real time on consumer chips. Remedies: integer networks [21], 8-bit phone models [13], [14] and integer-centric NVC (≈ −20% vs. VTM at 153.0/137.3 fps) [20], though NPU toolchains often reject INT16 [12]; calibration side streams [15], which give no guarantee in FP16 [12]; codebooks [16]; mixed fp16/FP32 precision across 12 GPUs [19]; and MLVC's *scale sending* [12]: scales are derived deterministically from the quantized hyperlatent (typically 128× fewer), enabling single-pass decoding (75→97 fps at 360p on an M3 Pro), with NPU-exact ReLU-family activations, gated memory, long-term references and periodic I-frames. MobileNVC [14] pipelines its *decoder* across a phone's NPU, GPU (parallel entropy coding, 11 ms with 512 threads vs. 16 ms with 8 CPU threads), warp unit and CPU, reaching 38.9 fps at 1080p; its encoder runs at about 3 fps. *Data*: MLVC uses VCD [41], HEVC B–E, UVG, MCL-JCV and a P.910 MOS test. *Performance*: MLVC −52.0% PSNR and −75.5% MOS BD-rate vs. HEVC-QSV; 33.8/33.1 fps at 720p on the M3 Pro Neural Engine; no catastrophic failure across Apple, Intel, Qualcomm and NVIDIA pairs [12]. *Limitations*: robustness costs compression (−52.0% vs. −69.6% for DCVC-RT); speeds are per direction, NPU only, with no power. The authors state that "simultaneous encoding and decoding on a single device at high resolutions (1080p and above) remains challenging" and that "power consumption is likely to become the next optimization frontier" [12].

E. Work built on DCVC-RT and MLVC

The real-time design is already being extended. UI^{2}C unifies intra and inter coding (−12.1% vs. DCVC-RT) [18]; integer-centric NVC makes it deterministic in integer arithmetic [20]; Streamable NVC turns it into a client–server codec across 12 GPUs and Wi-Fi, 5G and satellite links [19]; GVC-RT reuses a DCVC-RT-style encoder for generative ultra-low-bitrate coding [28]; and MLVC ports it to consumer NPUs [12]. All five run one direction on one processor; none studies concurrent coding or energy, which leaves the space this project targets.

F. Alternative neural families (considered, not selected)

Implicit representations (NeRV [24], C3 [25]) and Gaussian splatting [26] encode by per-video training, around 10^{-3} fps [11], so they cannot serve live calls. Generative codecs (GLC-Video [27], GVC-RT [28], 123.1/55.1 fps at 1080p) target perceptual metrics at ultra-low rate.

G. Heterogeneous parallel DNN execution on SoCs

Idea: use the CPU, GPU and NPU of one SoC at once, within a layer (μLayer [29], CoDL [30]), across branches (NN-Stretch [42]) or across concurrent models (Band [31]). MLPerf Mobile standardizes measurement [43]; Tu et al. measure DNN energy on edge devices [44]. *Performance*: CoDL up to 4.93× faster with 62.3% less energy; NN-Stretch up to 3.85×. On an M4 Pro the Neural Engine reaches 3.8 TFLOPS at about 5.2 W vs. 4.7 TFLOPS at 24 W for the GPU (preprint) [46], [47]. *Limitations*: stateless vision or language models on Android SoCs; co-execution often degrades under contention [45]; none handles a recurrent codec with bit-exact entropy scales, paired 33 ms deadlines and a CPU entropy coder in the loop.

III. From Classical to Real-Time Learned Coding: Pipelines and Hardware

A. How DCVC differs from a classical codec

Fig. 1(a)–(b) contrasts the two pipelines.

- *Prediction*: classical codecs subtract a block prediction and code the residual; DCVC feeds a learned feature context C_t to encoder and decoder and codes x_t conditioned on it [5].
- *Unit of coding*: fixed blocks (macroblocks, CTUs) vs. whole-frame latent tensors produced by convolutional networks; there is no block partitioning.
- *Transform*: fixed DCT/DST vs. a learned nonlinear transform at 1/8 or 1/16 resolution.
- *Entropy coding*: hand-designed CABAC contexts vs. a learned hyperprior [39] plus temporal prior that predicts each latent's Gaussian mean and scale for arithmetic coding.
- *Optimization*: the classical encoder searches modes and motion vectors per block (rate-distortion optimization), which is why VTM runs at 0.01 fps; a learned encoder is a fixed forward pass trained end to end on $\lambda R + D$, so its cost is constant and parallel, the property that makes real time possible.

TABLE I
HARDWARE AND SOFTWARE EACH CODEC WAS EVALUATED ON

Codec	Hardware tested	Best speed (enc/dec)	Software
HM, VTM [11]	CPU reference software	VTM 0.01/23.6 fps	C++
DCVC-DC, FM [10]	A100, RTX 4090	FM 5.0/5.9 (A100)	PyTorch, CUDA
DCVC-RT [10]	A100, RTX 4090, A6000, RTX 2080 Ti; EPYC CPU for entropy	125.2/112.8 (A100); 39.5/34.1 (2080 Ti)	PyTorch, CUDA, C++ rANS; int16 option
DCVC-UF [11]	RTX 4090, B200; tuned for 2080 Ti–B200	HT-S 1415.1/945.8 (B200)	PyTorch 2.9, CUDA 13, CUTLASS kernels
DHVC 2.0 [17]	RTX 3090; four GPUs for real time	≈4/7 fps on one GPU	PyTorch
MobileNVC [14]	Snapdragon 8 Gen 2 (NPU, GPU, warp unit, CPU)	38.9 fps decode	8-bit integer model
MLVC [12]	Apple M3 Pro Neural Engine, Intel Lunar Lake NPU, Snapdragon X Elite Hexagon	33.8/33.1 (720p, M3 Pro); MLVC-S 39.5/34.6 (1080p)	Core ML, ONNX Runtime; FP16

B. How learned coding became real time

Fig. 1(b)–(e) marks each change. DCVC to DCVC-FM kept optical-flow motion estimation and multi-scale contexts, many small layers that run at 3–6 fps [10]. DCVC-RT (c) removed motion entirely, learned one latent at 1/8 scale, cut module count and overlapped CPU arithmetic coding with GPU inference, reaching over 100 fps at 1080p [10]. DCVC-UF (d) codes eight frames as one latent, decodes them with parallel frame-specific decoders and reads the bitstream in one step [11]. MLVC (e) derives entropy scales deterministically from the hyperlatent, so any vendor's chip decodes the same stream, and the single decoding pass plus NPU-exact activations make it real time on laptop NPUs [12]. Speed therefore came from removing operations and synchronization points, not from cutting arithmetic.

C. Hardware used and minimum requirements

Table I lists the hardware each work was measured on. No paper states a formal minimum. The oldest GPU tested is the RTX 2080 Ti, where DCVC-RT still codes 1080p at 39.5/34.1 fps [10]. MLVC publishes speeds only for 2024 laptop NPUs, but its runtime library lists Apple M1–M5 Neural Engines as supported [48], so M1-class laptops can run it, likely at lower resolutions; training MLVC took 5 days on eight V100 GPUs [12], which is why we use the released checkpoints.

D. The DCVC family compared

Table II lists headline results on each paper's own anchor; Fig. 2 puts the family on one anchor. Test conditions shifted from GOP 10–12 to 96 frames with intra period 32 to all frames with intra period -1, and anchors from x265 to HM to VTM-17, so DCVC-FM reads -25.5% in its own paper but -21.3% when re-run with real fp16 bitstreams [9], [11]. Speed came from cutting operational cost, and every speed except MLVC's was measured on NVIDIA GPUs.

TABLE II
DCVC FAMILY: HEADLINE RESULTS ON EACH PAPER'S OWN ANCHOR

Model	Venue	Core idea	Headline result
DCVC [5]	NeurIPS'21	Conditional coding, flow	−26.0% vs. x265
TCM [6]	TMM'22	Multi-scale temporal contexts	14.4% vs. HM
HEM [7]	MM'22	Hybrid entropy model, variable rate	−18.2% vs. VTM (UVG)
DC [8]	CVPR'23	Diverse contexts, quadtree latent	−23.5% vs. HEM
FM [9]	CVPR'24	Feature modulation, 11.4 dB range	−25.5% vs. VTM-17
RT [10]	CVPR'25	No motion, one 1/8 latent, int16	−21.0%; 125/113 fps A100
UF HT-L [11]	CVPR'26	8-frame chunks, 120.5M params	−42.2% vs. VTM-LD
UF HT-S [11]	CVPR'26	Small chunk model, 81.2M	−31.6%; 1415/946 B200
UF LD [11]	CVPR'26	Chunk of 1, no delay, 9.7M	−9.5% vs. VTM-LD
MLVC [12]	arXiv'26	DCVC-RT + scale sending for NPUs	−75.5% MOS vs. QSV

IV. LITERATURE COMPARISON TABLE (8.3)

Table III summarizes the most relevant studies ("Std.": HEVC B–E, UVG, MCL-JCV).

V. CLOSEST EXISTING STUDIES (8.4)

Table IV details the four closest studies: MLVC is our baseline codec, DCVC-RT its parent, DCVC-UF the fastest family member, and MobileNVC the only codec already pipelined across a device's processors.

VI. SYNTHESIS OF THE LITERATURE (8.5)

Dominant approaches: conditional, motion-free learned coding [5], [10], [11] with hyperprior entropy models [39] made deterministic for deployment [12]. **Demonstrated**: compression beyond VVC (up to -42.2%), real time on GPUs (up to 655.9/453.3 fps), cross-vendor decoding on NPUs (≈33 fps per direction at 720p), a heterogeneous phone decoder [14] and large co-execution gains [30], [42]. **Performs well**: low-delay standard sequences, desktop GPUs, single-direction benchmarks. Table VI classifies the limitations.

Most relevant: the three evaluation gaps marked *core*, named by MLVC's own authors and closable with parallel-computing methods. **Directions considered and not chosen**: cross-chunk pipelining and content-adaptive chunk/model selection in DCVC-UF [11] (speed-only, the authors' own next step, fixed chunk size at test time); a GPU entropy coder [10] (partly done on phones [14], too long to build); fast deterministic kernels (crowded [12], [19], [20]); faster generative decoders [28]; error resilience [19] (thin parallel component); single-GPU DHVC 2.0 [17]; and faster NeRV encoding [24], [25] (crowded, unsuited to live calls; our original direction, dropped on this evidence); and a block-level complexity cascade (light and heavy models per block), rejected because learned codecs process whole frames, so regions cannot be routed to different models without padding artefacts. Frame-difference coding is subsumed by conditional coding (Section II-B).

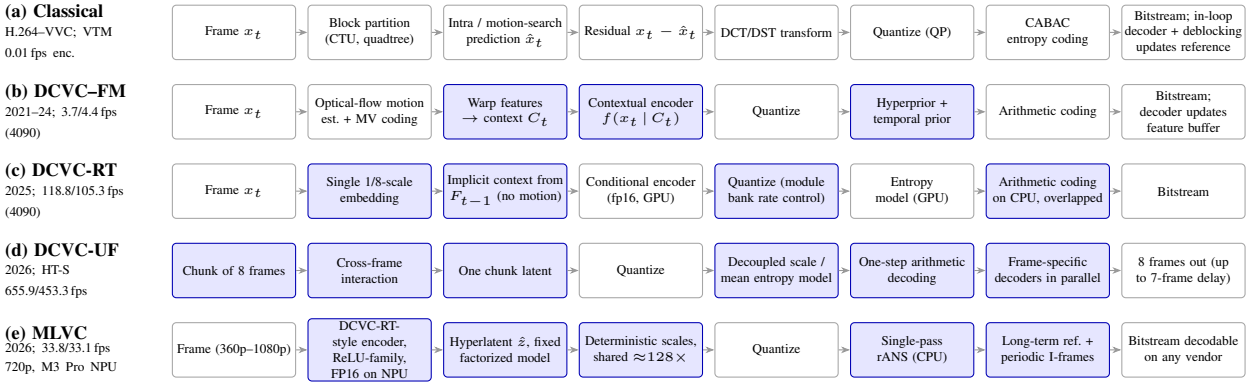


Fig. 1. Encoder pipelines from classical coding to MLVC (the decoder mirrors each). Shaded blocks are what changed relative to the row above: (b) conditional coding replaces residual coding; (c) motion is removed and entropy coding overlaps inference; (d) frames are coded in chunks; (e) scales become deterministic for cross-platform NPU decoding. Speeds are 1080p encode/decode unless noted [10]–[12].

TABLE III
COMPARISON OF THE MOST RELEVANT STUDIES

Study	Year	Problem	Dataset(s)	Method	Main innovation	Evaluation	Best results	Key limitation
VVC [3]	2021	Next-generation standard	JVET CTC	Hybrid block coding	Finer partitions, new tools	BD-rate, time	HM +42.4% bits vs. VTM (LD)	0.01 fps encode
DVC [4]	2019	End-to-end video codec	Vimeo; UVG, HEVC	Flow + residual autoencoders	All stages learned	PSNR, MS-SSIM	Beats H.264; matches H.265 (MS-SSIM)	Classical shape; slow
DCVC [5]	2021	Residual coding is suboptimal	Vimeo; Std.	Conditional coding	Feature-domain context	BD vs. x265	−26.0% (HEVC B)	Flow; far from real time
DCVC-FM [9]	2024	Narrow quality range	Vimeo; Std.	Feature modulation	Learnable quant. scaler	BD vs. VTM	−25.5% vs. VTM-17	5.0/5.9 fps A100
BiECVC [23]	2025	Trails VTM random access	Vimeo; Std.	Gated bidirectional contexts	Beats VTM-RA on all sets	BD vs. VTM-RA	−13.4% (IP 32)	RA delay; not real time
DCVC-RT [10]	2025	Too slow	Vimeo; Std.	Implicit temporal model, 1/8 latent	Operational cost sets speed	BD; 4 GPUs	−21.0% at 125.2/112.8 fps	NVIDIA only; int16 4× slower
UI^{2}C [18]	2026	Scene changes, drift	Std.	Unified intra/inter	One real-time model	BD vs. DCVC-RT	−12.1% vs. DCVC-RT	GPU only
DCVC-UF [11]	2026	Frame-wise coding caps speed	Vimeo; Std.	Chunk coding	One-step entropy decoding	BD; 5 GPUs	−42.2% at 371/274 fps	Fixed chunk; no pipelining; delay
MobileCodec [13]	2022	Neural codec on a phone	Std.	8-bit model on NPU	First phone inter decoding	Speed, BD	720p phone decoding	+190.9% vs. QSV [12]
MobileNVC [14]	2024	1080p on a phone	HEVC B, others	Decoder over NPU, GPU, warp, CPU	Heterogeneous decoder pipeline	fps, stage latency	38.9 fps 1080p decode	Decoder only; weak RD
Tian [15]	2023	Cross-platform failures	Std.	Calibration side stream	Mismatch correction	BD, fps	25 fps 720p; up to −24.2% vs. H.265	No guarantee in FP16
Int.-centric [20]	2026	Determinism	Std.	Integer arithmetic	Integer real-time codec	BD, fps	≈ −20% at 153/137 fps	GPUs, not NPUs
Streamable [19]	2026	Cross-GPU streaming	1080p seq.	fp16 P / FP32 I	Mixed-precision resync	Cross-GPU, network	12 GPUs decode	GPUs only; preprint
MLVC [12]	2026	Cross-chip failure, heavy models	Vimeo; VCD, Std.	Scale sending, exact activations	Robust real-time NPU codec	BD vs. QSV; 3 NPUs	−75.5% MOS; 33.8/33.1 fps 720p	Per direction; NPU only; no power
CoDL [30]	2022	Slow mobile DNNs	Vision models	CPU–GPU co-execution	Latency predictor	Latency, energy	4.93×, −62.3% energy	Stateless, Android
Band [31]	2022	Concurrent DNNs	Multi-DNN apps	Subgraph scheduling	CPU/GPU/DSP/NPU scheduler	Latency	Concurrent models on 4 processor types	No shared state or deadlines
NN-Stretch [42]	2023	One processor at a time	Vision models	Network branching	Parallel branches	Latency	3.85× vs. one processor	Changes model
Tu et al. [44]	2023	Unmeasured DNN energy	Energy datasets	Measurement, prediction	Cross-device energy scoring	Energy	Energy datasets, predictors	No codecs

VII. Research Gap (8.6)

No published learned video codec has been run with its encoder and decoder working at the same time on one consumer device, and none reports how placing its parts on the CPU, GPU and NPU changes throughput, latency, energy per frame or decoding correctness. Three findings support it. (1) *Concurrency*: MLVC runs at 33.8 and 33.1 fps at 720p on the M3 Pro Neural Engine separately [12]; sharing it sequentially gives about 59.8 ms per frame pair, ≈16.7 fps per direction (our estimate). (2) *Placement*: MLVC decodes correctly on all processors but reports NPU speed only; heterogeneous pipelining exists only for a phone decoder [14]. (3) *Energy*: unreported for every learned codec [12].

TABLE IV
THE FOUR CLOSEST STUDIES AND HOW THIS PROJECT DIFFERS

	MLVC [12] (2026)	DCVC-RT [10] (CVPR'25)	DCVC-UF [11] (CVPR'26)	MobileNVC [14] (WACV'24)
Problem	Cross-chip decoding failures; models too heavy for devices	What really limits learned-codec speed	Frame-wise coding caps speed and compression	Real-time 1080p decoding on a phone
Method	DCVC-RT + scale sending, ReLU-family activations, gated memory, LTR; FP16 on NPUs	Implicit temporal model, one 1/8 latent, int16 mode, CPU entropy coding overlapped	8-frame chunks, frame-specific decoders, one-step entropy decoding	8-bit model; decoder pipelined over NPU, GPU, warp unit, CPU
Datasets	Vimeo-90k; VCD, HEVC B–E, UVG, MCL-JCV	Vimeo-90k; HEVC B–E, UVG, MCL-JCV	Same, 128-frame fine-tuning	Incl. HEVC Class B
Contribution	First robust, real-time, competitive codec on consumer NPUs	Operational cost, not MACs, limits speed	Best compression at highest throughput	First real-time neural decoder on a phone
Evaluation	PSNR/MS-SSIM/LPIPS/MOS BD vs. QSV; NPU fps on Apple, Intel, Qualcomm	BD vs. HM/VTM/ECM; fps on 4 GPUs	BD vs. VTM LD/RA; fps on 5 GPUs	BD vs. x264/x265; per-stage latency
Results	−52.0% PSNR, −75.5% MOS; M3 Pro 129.5/122.9 (360p), 33.8/33.1 (720p), 15.6/15.2 fps (1080p)	−21.0% at 125.2/112.8 fps (A100)	HT-L −42.2% at 371.1/273.6; HT-S −31.6% at 655.9/453.3 fps	38.9 fps decode; encoder ≈3 fps
Limitations	Directions timed separately; NPU only; no power	NVIDIA only; fp16 not portable; int16 slow	Fixed chunk; no cross-chunk pipelining; up to 7-frame delay	Decoder only; weak compression
We differ	**Run both directions at once; measure all processors and energy; schedule placement**	**Laptop SoC; overlap across call directions, not within a frame**	**Zero-delay calling on consumer devices**	**Both directions of a cross-platform-safe codec, with energy**

TABLE V
NOVELTY COMPARISON WITH THE CLOSEST STUDIES

Feature / capability	MLVC [12]	MobileNVC [14]	DCVC-RT [10]	Band [31]	**Proposed (DuplexNVC)**
Encoder + decoder at once on one device	No, timed separately	No, decoder only	No	Not a codec	**Yes, full duplex**
Placement over CPU, GPU, NPU	NPU speed only	Decoder only	CPU entropy beside GPU	Stateless models	**Both directions, codec-aware**
Pipelining	None reported	$\lesssim$3 frames in decoder	Within a frame (+12%/+9%)	Subgraph level	**Across directions (t+1 enc., t dec.)**
Energy per frame	No	No	No	No	**Per processor (powermetrics)**
Cross-processor correctness	Yes (scale sending)	Not tested	int16 only	N/A	**Every placement and across machines**
Cross-condition evaluation	3 NPU vendors, 4 res.	One phone	4 NVIDIA GPUs	Several phones	**4 res. × 2 models × 2 Mac gens × all placements**
Proposed mechanism	Scale sending	Integer model + decoder pipeline	Implicit temporal model	Multi-DNN scheduler	**Energy-aware placement, model and resolution controller**

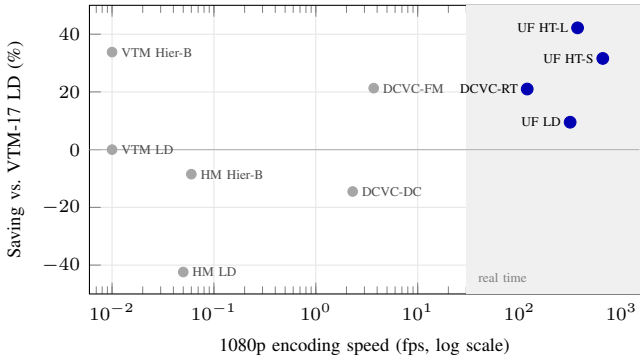


Fig. 2. Only DCVC-RT and DCVC-UF both beat VTM and encode 1080p above 30 fps. Data: DCVC-UF Table 1 [11] (YUV420, all frames, intra period −1; neural codecs on an RTX 4090).

TABLE VI
LIMITATIONS, THEIR TYPE AND RELEVANCE

Limitation (evidence)	Type	Relevant
Encoder/decoder speed reported separately [12]	Evaluation, practical	**Core**
Speed on the NPU only [12]	Evaluation	**Core**
No energy figures for any learned codec [10]–[12], [14]	Evaluation	**Core**
Heterogeneous pipeline only for a phone decoder [14]	Computational	Yes
Co-execution ignores codec constraints [29]–[31]	Methodological	Yes
Robustness costs compression [12]	Methodological	Context
Weak above 40 dB [10], [11]	Methodological	No
Fixed chunks, no cross-chunk pipelining [11]	Computational	No
Anchor-dependent BD-rates [5], [9], [12]	Evaluation	Yes
Released checkpoints differ from paper [40]	Data	Yes

VIII. PROPOSED INNOVATIVE ADVANTAGE (8.7)

We propose **DuplexNVC**, a scheduler that runs both call directions at once on an Apple Silicon laptop. It places MLVC's encoder network, decoder network and rANS entropy coders on the CPU, GPU and Neural Engine, pipelines them, and selects model (MLVC or MLVC-S) and resolution to hold 30 fps each way at minimum energy without breaking decoding (Fig. 3).

Mechanism. (1) *Profiler*: per-stage latency and per-processor power for every model, resolution and processor. (2) *Placement*: networks pinned via Core ML compute units (CPU, CPU+GPU, CPU+Neural Engine); rANS coders on CPU worker threads. (3) *Pipeline*: three stages per direc-

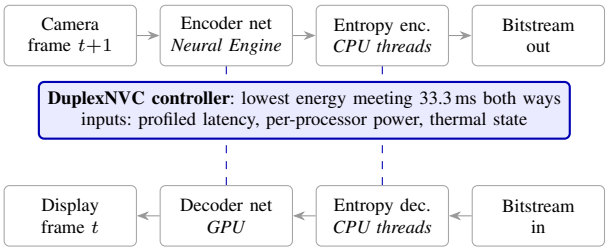


Fig. 3. Proposed pipeline (one candidate placement): the device encodes frame $t{+}1$ while decoding remote frame t; the controller moves networks between processors and switches model and resolution.

TABLE VII
RESEARCH QUESTIONS AND TESTABLE HYPOTHESES

	Question	Hypothesis
RQ1	Speed and power of each MLVC stage on each processor?	Neural Engine lowest J/frame; GPU competitive at $\geq$720p
RQ2	Loss when both directions share the Neural Engine?	$\approx$17 fps per direction at 720p on M3-class (H1)
RQ3	Does heterogeneous placement + pipelining restore real time?	$\geq$1.5$\times$ full-duplex throughput at 720p (H2)
RQ4	Does the controller beat fixed configurations?	30 fps both ways one resolution step higher at $\leq$ baseline energy (H3)
RQ5	Is decoding still correct across processors?	Zero failures; $<$1 BD-rate point change (H4)

tion with double-buffered queues. (4) *Controller*: picks the minimum-energy configuration meeting the 33.3 ms budget in both directions and re-checks every N frames. A failed hypothesis is still a finding (e.g., if the Neural Engine cannot serve two models concurrently). *Lessons reused from earlier models*: per-processor pipelining from MobileNVC's decoder [14], CPU entropy coding overlapped with accelerator inference from DCVC-RT [10], multi-model scheduling from Band [31], latency prediction from CoDL [30], and scale sending, which keeps decoding correct when a network moves between processors [12].

Evaluation. Baselines: B0, MLVC encoder then decoder on the Neural Engine; B1, both on the Neural Engine in two naive threads; B2, Apple hardware HEVC (VideoToolbox). Compression metrics: BD-rate in PSNR, MS-SSIM and VMAF vs. B2 and single-processor MLVC; decoding correctness across processors and machines. Parallel-performance metrics: full-duplex fps, median/95th-percentile latency, speedup over B0, utilization, queue and synchronization overhead, energy per frame from `powermetrics` (CPU, GPU, Neural Engine separately), and 10-minute thermal runs. Operating points: 360p–1080p; MLVC and MLVC-S; four rates; M1- and M3/M4-class Macs. Data: HEVC B–E, UVG, MCL-JCV, VCD [41] (Table VII). *Risks*: libmlvc fully supports only NPU execution (CPU/GPU experimental) [48]; fallback is ONNX Runtime on CPU and NVIDIA GPU.

IX. Novelty Comparison (8.8)

Only the proposed method combines concurrent coding, three-processor placement and energy measurement (Table V). Expected contributions: (C1) the first per-processor speed and energy characterization of a cross-platform learned codec on a laptop SoC; (C2) the first full-duplex evaluation of a learned codec on one consumer device; (C3) DuplexNVC, a pipelined, energy-aware scheduler that keeps both directions in real time with correct decoding.

X. Project Approval Gate (8.9)

What is innovative? MLVC [12] timed each direction alone, NPU only, without energy; MobileNVC [14] pipelined only a phone decoder; co-execution systems [29]–[31] schedule stateless networks. We are the first to run both directions of a learned-codec call at once on one laptop, measure every processor's speed and energy, and schedule placement, model and resolution for 30 fps each way at minimum energy.

References

[1] T. Wiegand, G. J. Sullivan, G. Bjøntegaard, and A. Luthra, "Overview of the H.264/AVC video coding standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 7, pp. 560–576, Jul. 2003.

[2] G. J. Sullivan, J.-R. Ohm, W.-J. Han, and T. Wiegand, "Overview of the High Efficiency Video Coding (HEVC) standard," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 22, no. 12, pp. 1649–1668, Dec. 2012.

[3] B. Bross *et al.*, "Overview of the Versatile Video Coding (VVC) standard and its applications," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 10, pp. 3736–3764, Oct. 2021.

[4] G. Lu, W. Ouyang, D. Xu, X. Zhang, C. Cai, and Z. Gao, "DVC: An end-to-end deep video compression framework," in *Proc. IEEE/CVF CVPR*, 2019, pp. 11006–11015.

[5] J. Li, B. Li, and Y. Lu, "Deep contextual video compression," in *Proc. NeurIPS*, vol. 34, 2021.

[6] X. Sheng, J. Li, B. Li, L. Li, D. Liu, and Y. Lu, "Temporal context mining for learned video compression," *IEEE Trans. Multimedia*, vol. 25, 2023, doi: 10.1109/TMM.2022.3220421.

[7] J. Li, B. Li, and Y. Lu, "Hybrid spatial-temporal entropy modelling for neural video compression," in *Proc. ACM Int. Conf. Multimedia*, 2022.

[8] J. Li, B. Li, and Y. Lu, "Neural video compression with diverse contexts," in *Proc. IEEE/CVF CVPR*, 2023.

[9] J. Li, B. Li, and Y. Lu, "Neural video compression with feature modulation," in *Proc. IEEE/CVF CVPR*, 2024.

[10] Z. Jia, B. Li, J. Li, W. Xie, L. Qi, H. Li, and Y. Lu, "Towards practical real-time neural video compression," in *Proc. IEEE/CVF CVPR*, 2025, pp. 12543–12552.

[11] J. Li, W. Xie, Z. Jia, B. Li, Z. Guo, X. Zhang, and Y. Lu, "Ultra-fast neural video compression," in *Proc. IEEE/CVF CVPR*, 2026, arXiv:2606.04410.

[12] T. Pärnamaa, M. Lumiste, A. Loot, E. Indenbom, A. Znobishchev, and A. Saabas, "MLVC: A multi-platform learned video codec for real-world deployment," arXiv:2606.28027, 2026.

[13] H. Le *et al.*, "MobileCodec: Neural inter-frame video compression on mobile devices," in *Proc. ACM MMSys*, 2022, pp. 324–330.

[14] T. van Rozendaal *et al.*, "MobileNVC: Real-time 1080p neural video compression on a mobile device," in *Proc. IEEE/CVF WACV*, 2024, pp. 4311–4321.

[15] K. Tian, Y. Guan, J. Xiang, J. Zhang, X. Han, and W. Yang, "Towards real-time neural video codec for cross-platform application using calibration information," in *Proc. ACM Int. Conf. Multimedia*, 2023, pp. 7961–7970.

[16] K. Tian *et al.*, "Effortless cross-platform video codec: A codebook-based method," arXiv:2310.10292, 2023.

[17] M. Lu, Z. Duan, W. Cong, D. Ding, F. Zhu, and Z. Ma, "High-efficiency neural video compression via hierarchical predictive learning," arXiv:2410.02598, 2024.

[18] H. Xiang, Y. Bian, L. Li, J. Wu, X. Zhang, and D. Liu, "Real-time neural video compression with unified intra and inter coding," in *Proc. IEEE/CVF CVPR*, 2026, pp. 35217–35226.

[19] K. Arunruangsirilert, H. Sun, and J. Katto, "Streamable neural video compression: A mixed precision approach for cross-platform deployment," arXiv:2608.00483, 2026.

[20] Z. Jia, W. Xie, Z. Guo, B. Li, J. Li, H. Li, and Y. Lu, "Integer-centric neural video compression," in *Proc. ICLR*, 2026.

[21] J. Ballé, N. Johnston, and D. Minnen, "Integer networks for data compression with latent-variable models," in *Proc. ICLR*, 2019.

[22] X. Sheng, L. Li, D. Liu, and S. Wang, "Bi-directional deep contextual video compression," *IEEE Trans. Multimedia*, vol. 27, pp. 5632–5646, 2025.

[23] W. Jiang, J. Li, K. Zhang, and L. Zhang, "BiECVC: Gated diversification of bidirectional contexts for learned video compression," in *Proc. ACM Int. Conf. Multimedia*, 2025, pp. 7248–7257.

[24] H. Chen, B. He, H. Wang, Y. Ren, S.-N. Lim, and A. Shrivastava, "NeRV: Neural representations for videos," in *Proc. NeurIPS*, vol. 34, 2021, pp. 21557–21568.

[25] H. Kim, M. Bauer, L. Theis, J. R. Schwarz, and E. Dupont, "C3: High-performance and low-complexity neural compression from a single image or video," in *Proc. IEEE/CVF CVPR*, 2024, pp. 9347–9358.

[26] I. Lee, Y. Choi, and J. Lee, "GaussianVideo: Efficient video representation and compression by Gaussian splatting," in *Proc. IEEE/CVF CVPR Workshops*, 2025, pp. 4510–4519.

[27] L. Qi, Z. Jia, J. Li, B. Li, H. Li, and Y. Lu, "Generative latent coding for ultra-low bitrate image and video compression," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 10, pp. 10500–10515, 2025.

[28] T. Dang, S. Wang, L. Luo, G. Lu, and J. Dai, "GVC-RT: Towards real-time generative video compression at ultra-low bitrates," arXiv:2608.04891, 2026.

[29] Y. Kim, J. Kim, D. Chae, D. Kim, and J. Kim, "μLayer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization," in *Proc. EuroSys*, 2019, pp. 1–15.

[30] F. Jia *et al.*, "CoDL: Efficient CPU-GPU co-execution for deep learning inference on mobile devices," in *Proc. ACM MobiSys*, 2022, pp. 209–221.

[31] J. S. Jeong *et al.*, "Band: Coordinated multi-DNN inference on heterogeneous mobile processors," in *Proc. ACM MobiSys*, 2022, pp. 235–247.

[32] G. Bjøntegaard, "Calculation of average PSNR differences between RD-curves," ITU-T SG16/Q6 VCEG, Doc. VCEG-M33, Apr. 2001.

[33] Z. Wang, E. P. Simoncelli, and A. C. Bovik, "Multiscale structural similarity for image quality assessment," in *Proc. Asilomar Conf. Signals, Syst. Comput.*, 2003, pp. 1398–1402.

[34] Z. Li, A. Aaron, I. Katsavounidis, A. Moorthy, and M. Manohara, "Toward a practical perceptual video quality metric," Netflix Technology Blog, Jun. 2016.

[35] A. Mercat, M. Viitanen, and J. Vanne, "UVG dataset: 50/120fps 4K sequences for video codec analysis and development," in *Proc. ACM MMSys*, 2020, pp. 297–302.

[36] H. Wang *et al.*, "MCL-JCV: A JND-based H.264/AVC video quality assessment dataset," in *Proc. IEEE ICIP*, 2016, pp. 1509–1513.

[37] T. Xue, B. Chen, J. Wu, D. Wei, and W. T. Freeman, "Video enhancement with task-oriented flow," *Int. J. Comput. Vis.*, vol. 127, no. 8, pp. 1106–1125, 2019.

[38] F. Bossen, "Common test conditions and software reference configurations," JCT-VC, Doc. JCTVC-L1100, Jan. 2013.

[39] J. Ballé, D. Minnen, S. Singh, S. J. Hwang, and N. Johnston, "Variational image compression with a scale hyperprior," in *Proc. ICLR*, 2018.

[40] Microsoft, "MLVC repository and released checkpoints," GitHub. [Online]. Available: https://github.com/microsoft/mlvc

[41] B. Naderi *et al.*, "VCD: A video conferencing dataset for video compression," in *Proc. IEEE ICASSP*, 2024.

[42] J. Wei *et al.*, "NN-Stretch: Automatic neural network branching for parallel inference on heterogeneous multi-processors," in *Proc. ACM MobiSys*, 2023, pp. 70–83.

[43] V. Janapa Reddi *et al.*, "MLPerf Mobile inference benchmark," in *Proc. MLSys*, vol. 4, 2022, pp. 352–369.

[44] X. Tu *et al.*, "Unveiling energy efficiency in deep learning: Measurement, prediction, and scoring across edge devices," in *Proc. ACM/IEEE SEC*, 2023.

[45] S. Liu *et al.*, "Deep learning inference on heterogeneous mobile processors: Potentials and pitfalls," in *Proc. AdaAIoTSys Workshop (MobiSys)*, 2024, pp. 1–6.

[46] P. Hübner, A. Hu, I. Peng, and S. Markidis, "Apple vs. Oranges: Evaluating the Apple Silicon M-series SoCs for HPC performance and efficiency," arXiv:2502.05317, 2025.

[47] Á. Corrochano López and C. García Sánchez, "Evaluation of domain-specific architectures for general-purpose applications in Apple Silicon," arXiv:2511.13450, 2025.

[48] Microsoft, "libmlvc: runtime library for MLVC," GitHub. [Online]. Available: https://github.com/microsoft/libmlvc